

CareerMatch AI – Python Developer Role Guide

Synthetic/Generalized Career Knowledge – Hackathon Prototype

1. Role Overview

A Python Developer builds backend applications, scripts, and services using Python, often combined with a web framework and a database. This is generalized educational content for a hackathon prototype.

2. Responsibilities

- Write clean, maintainable Python code for applications or services
- Build and maintain APIs that other systems can consume
- Work with databases to store and retrieve application data
- Write tests and debug issues in existing code
- Collaborate using version control in a team setting

3. Required Skills

Python

What it is: A general-purpose, beginner-friendly programming language widely used for data work, automation, and AI development.

Why it matters: It is the primary language for machine learning, data analysis, and AI tooling because of its simple syntax and large ecosystem of libraries.

How it's used in this role: Used to load data, build models, write automation scripts, and connect components of an AI pipeline.

Beginner expectation: Can write scripts using variables, loops, conditionals, functions, and lists/dictionaries.

OOP

What it is: Object-Oriented Programming: a paradigm that organizes code around objects containing data and behavior.

Why it matters: It is the standard way of structuring maintainable, reusable code in most professional codebases.

How it's used in this role: Used to structure larger applications in a maintainable way.

Beginner expectation: Understands classes, objects, and basic inheritance.

Data Structures

What it is: Ways of organizing data such as arrays, lists, stacks, queues, trees, and graphs.

Why it matters: Efficient data structures are essential for writing performant, scalable code.

How it's used in this role: Used to organize data efficiently within any software system.

Beginner expectation: Understands arrays, lists, stacks, and queues and their basic operations.

Algorithms

What it is: Step-by-step procedures for solving computational problems efficiently.

Why it matters: Strong algorithmic thinking is core to technical interviews and efficient software design.

How it's used in this role: Used to design efficient solutions and is heavily tested in coding interviews.

Beginner expectation: Understands sorting, searching, and basic time complexity.

Exception Handling

What it is: Techniques for detecting and responding to errors that occur while a program runs.

Why it matters: Robust error handling prevents applications from crashing unexpectedly in production.

How it's used in this role: Used to make applications resilient to unexpected input or failures.

Beginner expectation: Can use try/except blocks to handle simple errors.

File Handling

What it is: Techniques for reading from and writing to files programmatically.

Why it matters: Many real applications need to read input files or write output logs and results.

How it's used in this role: Used to load datasets or configuration files into a program.

Beginner expectation: Can read and write simple text or CSV files.

APIs

What it is: Interfaces that allow different software systems to communicate with each other.

Why it matters: APIs connect models, data sources, and applications into a working system.

How it's used in this role: Used to connect a trained model to a frontend or another application.

Beginner expectation: Understands what an API is and can call a simple public API.

4. Backend Development

FastAPI

What it is: A modern Python web framework used to build APIs quickly.

Why it matters: It is a common choice for exposing machine learning models or backend logic as a web service.

How it's used in this role: Used to serve a trained model or application logic over the web.

Beginner expectation: Can build a simple API endpoint that returns data.

Flask

What it is: A lightweight Python web framework used to build web applications and APIs.

Why it matters: It is a simple, flexible option for building small backend services.

How it's used in this role: Used to build lightweight backend services or model-serving APIs.

Beginner expectation: Can build a simple web route that returns data.

5. Databases

SQL

What it is: A query language used to read, filter, and aggregate data stored in relational databases.

Why it matters: Most real-world data lives in databases, so extracting and shaping it correctly is a foundational skill for any data-related role.

How it's used in this role: Used to pull structured datasets from a company database before analysis or model training.

Beginner expectation: Can write SELECT, WHERE, GROUP BY, and simple JOIN queries.

PostgreSQL

What it is: An open-source relational database management system.

Why it matters: It is one of the most widely used production-grade relational databases.

How it's used in this role: Used to store and query structured application or analytical data.

Beginner expectation: Can create tables and run basic queries.

MySQL

What it is: A popular open-source relational database management system.

Why it matters: It is widely used in web applications and is a common choice for learning relational databases.

How it's used in this role: Used to store structured data for applications and analytics.

Beginner expectation: Can create tables and run basic queries.

6. Git/GitHub

Git

What it is: A version control system used to track changes in code over time.

Why it matters: It is essential for collaborating on code and maintaining a reliable project history.

How it's used in this role: Used to manage and share project code, including hackathon submissions.

Beginner expectation: Can commit changes, create branches, and push to a remote repository.

7. Testing

Testing

What it is: The practice of verifying that code behaves correctly through automated or manual checks.

Why it matters: Testing catches bugs early and gives confidence that changes do not break existing functionality.

How it's used in this role: Used to validate that code and models work as expected before release.

Beginner expectation: Can write simple unit tests for individual functions.

8. Deployment

Deployment

What it is: The process of releasing software so it can be used in a live environment.

Why it matters: Code only creates value once it is deployed and accessible to real users.

How it's used in this role: Used to release applications so they can be accessed by users.

Beginner expectation: Can deploy a simple application to a basic hosting service.

9. Education

Commonly pursued by graduates in Computer Science, Information Technology, or related fields, though many strong Python developers are self-taught through projects and open-source work.

10. Fresher Expectations

- Writes clean, readable Python following basic best practices
- Can build a simple API endpoint using FastAPI or Flask
- Comfortable connecting an application to a basic database
- Uses Git for version control on personal or team projects
- These expectations describe a general, entry-level readiness bar used only for hackathon prototyping. They are illustrative and not tied to any real company's actual hiring bar, vacancy, or deadline.

11. High/Medium/Low Priority Skills

High Priority	Medium Priority	Low Priority
• Python • Data Structures • APIs	• OOP • SQL • FastAPI/Flask • Git	• Testing • Deployment • Exception Handling

12. Common Skill Gaps

- Comfortable with scripting but limited experience structuring larger applications
- Little exposure to writing automated tests
- Limited experience with real database integration beyond toy examples
- Unfamiliarity with deploying an application outside a local machine

13. Portfolio Projects

1. Local Business Directory API

Problem: Build a REST API that lists and searches small local businesses.

Skills used: FastAPI, SQL, APIs

Difficulty: Beginner to Intermediate

2. Personal Finance Tracker Backend

Problem: Build a backend service to record and summarize personal expenses.

Skills used: Python, OOP, Databases, Testing

Difficulty: Intermediate

14. Interview Topics

- Core Python data structures and their time complexity
- Object-oriented design principles
- REST API design
- Working with relational databases
- Debugging strategies

15. Coding Questions

- 1 Reverse a string without using built-in reverse functions.
- 2 Find duplicate elements in a list efficiently.
- 3 Implement a function that checks if a string is a palindrome.
- 4 Given a list of orders, group and total them by customer using a dictionary.
- 5 Write a simple API endpoint that returns a list of items from a database.

16. Learning Roadmap

- 1 Python fundamentals and OOP
- 2 Data structures and algorithms
- 3 Working with files and exceptions
- 4 SQL and databases
- 5 Building APIs with FastAPI or Flask
- 6 Git and basic testing
- 7 Deployment basics

17. RAG Retrieval Keywords

Python developer, FastAPI, Flask, REST API, backend development, entry-level Python developer

Document Metadata

Job ID	PY001
Role	Python Developer
Category	Software Development
Experience Level	Entry Level
Data Status	Synthetic/Generalized